

Turtlesim Node

Proyecto primer corte

Nicol Juliana Tuta Niño
Escuela Colombiana de Ingeniería
Julio Garavito
Bogotá, Colombia
nicol.tuta-n@mail.escuelaing.edu.co

Ángel Fabián Pedreros Almeida
Escuela Colombiana de Ingeniería
Julio Garavito
Bogotá, Colombia
angel.pedreros-a@mail.escuelaing.edu.co

I. METODOLOGÍA

La arquitectura del sistema desarrollado en el repositorio se ha implementado una distinción clara entre el paquete de interfaces y el nodo de control para poder realizar la tarea deseada.

paquete de interfaces y paquete de nodos

A. *turtle_interface*

- **Servicio:** Se realizó un servicio estructurado con strings para cambiar el modo de la tortuga de manera sincrónica y controlada, pero sin la necesidad de realizar un feedback, ni de requerir de mucho tiempo. Al ser una comunicación tipo *Request/Response* se agregó a la estructura un bool de success confirmando que el estado de la tortuga fue actualizado correctamente.
- **Acción:** La acción Trajectory fue implementada para gestionar el movimiento de múltiples puntos definidos por el usuario. Se usó el tipo de estructura de mensaje *geometry_msgs/Points* y se diseñó para obtener un feedback constante así como la posibilidad de poder cancelar el proceso en cualquier momento por el usuario. Se tiene tres secciones en la estructura, los *Goals* que son los puntos a alcanzar, *result* que se dio como un booleano que indicara si fue exitoso o no y por último un *feedback* dado por un string que indicara en que proceso del movimiento está en todo momento.

B. *turtle_control*

Para el nodo de control se toma un tipo de comunicación *subscriber-publisher*, para este caso el nodo de control actúa tanto como suscriptor como publicador de dos tópicos:

- **turtlesim/msg/Pose:** El nodo actúa como suscriptor del tópico */turtle1/pose* que transmite continuamente la posición y orientación actual de la tortuga mediante el mensaje *turtlesim/msg/Pose*, permitiendo que el nodo conozca su estado en tiempo real. Esta suscripción es fundamental para implementar el control, ya que el cálculo de velocidades depende del error entre la posición actual y el punto objetivo.
- **turtlesim/msg/Pose:** El nodo publica mensajes de tipo *geometry_msgs/msg/Twist* en el tópico, el cual controla directamente la velocidad lineal y angular de la tortuga.

En cada parte del algoritmo, tanto en el movimiento de circunferencia como en el camino de trayectoria se da el cálculo de un error para ajustar dichas velocidades. En caso de la circunferencia un error del ángulo y en caso de la trayectoria, entre la posición deseada y la actual, para poder disminuir la velocidad cuando este cerca de llegar.

Este enfoque aprovecha la naturaleza asíncrona de ROS 2: la suscripción actualiza el estado en tiempo real, mientras que la publicación envía comandos de forma continua. Gracias a esta arquitectura distribuida, el sistema mantiene modularidad, escalabilidad y separación clara entre percepción (pose), decisión (control_node) y actuación (cmd_vel). Se proporciona la imagen de como **RQT_GRAPHICS** muestra dicha comunicación descrita previamente.

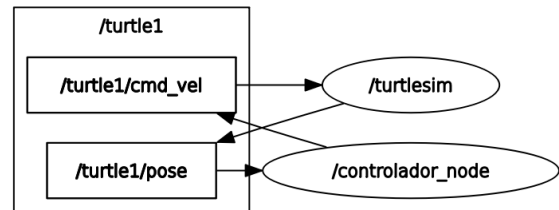


Fig. 1. RQT graph

C. *Compilación de los paquetes*

1) *Paquete de Interfaz:* El archivo CMakeLists.txt del paquete *turtle_interfaces* tiene como objetivo principal generar las interfaces personalizadas del sistema (servicios y acciones) a partir de los archivos definidos *ChangeMode.srv* y *Trajectory.action*. A diferencia de un paquete de nodos en Python, este paquete no contiene lógica de ejecución, sino definiciones estructurales que serán utilizadas por otros nodos. Para ello, se emplea el sistema de construcción *ament_cmake*, que es el estándar en ROS 2 para paquetes basados en CMake.

En este archivo se importaron distintas dependencias necesarias para poder lograr la trayectoria y que el nodo pueda ser utilizado por otros nodos de ROS2. Para ello, se realizó una exportación de dependencias de la librería *rosidl_default_generators*, y de *geometry_msgs* como se muestra el siguiente fragmento del CMake:

```

find_package(geometry_msgs REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "srv/ChangeMode.srv"
  "action/Punto.action"
  DEPENDENCIES geometry_msgs
)

```

Fig. 2. CMake Dependencies

Por otro lado, mientras que el *CMakeLists.txt* se encarga del proceso de compilación, el *package.xml* define formalmente el nombre del paquete, su versión, mantenedor, licencia y, especialmente, sus dependencias. Este archivo permite que herramientas como colcon y el sistema de resolución de dependencias de ROS 2 comprendan cómo integrar el paquete dentro del workspace. Al igual que en el *Cmake* se debe declarar como herramienta de construcción del nodo, las librerías de *ament_cmake*, *rosidl_default_generators*.

```

<buildtool_depend>ament_cmake</buildtool_depend>

<depend>geometry_msgs</depend>
<buildtool_depend>rosidl_default_generators</buildtool_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>

```

Fig. 3. Archivo package.xml

Es importante mencionar que se genera un *member_group* para lograr que ROS2 detecte nuestro nodo de interfaz como parte de la lista de interfaces contenidas en el workspace. Sin esta línea en el archivo XML, la interfaz no podría ser utilizada por otros nodos. Primero se realiza *colcon* con este paquete debido a que nuestro nodo de control depende del servicio y acción definidas en este paquete, por ende es necesario en el siguiente paso incluirlo como una librería con dependencia.

2) *Paquete de Control*: Para ejecutar el sistema se necesita del archivo *setup.py* del paquete *turtle_control* que es el encargado de definir cómo se instala y registra el paquete Python dentro de ROS 2, permitiendo que el nodo pueda ejecutarse mediante el comando *ros2 run*. En este archivo se declara el nombre del paquete, se indican las carpetas Python que deben incluirse mediante *find_packages()*, se registran los archivos necesarios para que ROS 2 lo reconozca, y se especifican las dependencias básicas de instalación; sin embargo, la parte más importante es *entry_points*, donde se define el ejecutable *controller_node* y se enlaza con la función *main()* del archivo *controller_node.py*. Gracias a esta configuración, cuando se compila con *colcon build*, el sistema instala el paquete y crea el ejecutable, de modo que al ejecutar *ros2 run turtle_control controller_node*, ROS 2 localiza el paquete, identifica el eje-

cutable declarado y lanza automáticamente la función *main()*, iniciando así el nodo de control y permitiendo que comience la comunicación mediante publicadores, suscriptores, servicios y acciones.

Por otra parte, en el *package.xml* se declaran las mismas dependencias pero clasificadas según su rol: *buildtool_depend* para *ament_cmake*, *depend* para *rcplpy*, *geometry_msgs*, *turtlesim* y *turtle_interfaces*, asegurando que tanto en compilación como en ejecución el sistema sepa qué librerías necesita. En conjunto, estos dos archivos permiten que el nodo de control pueda comunicarse mediante publicadores, suscriptores, servicios y acciones, integrándose correctamente con el ecosistema de ROS 2 y garantizando que todas las dependencias estén enlazadas y disponibles al momento de ejecutar el sistema.

```

<depend>rcplpy</depend>
<depend>geometry_msgs</depend>
<depend>turtlesim</depend>
<depend>turtle_interface</depend>
<depend>rcplpy_action</depend>

```

Fig. 4. Archivo package.xml del nodo de control

II. RESULTADOS

A. Modo Manual

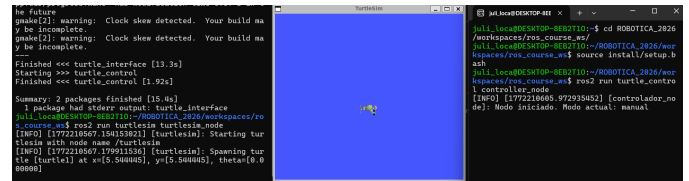


Fig. 5. Modo manual

B. Modo Circunferencia

Antihorario

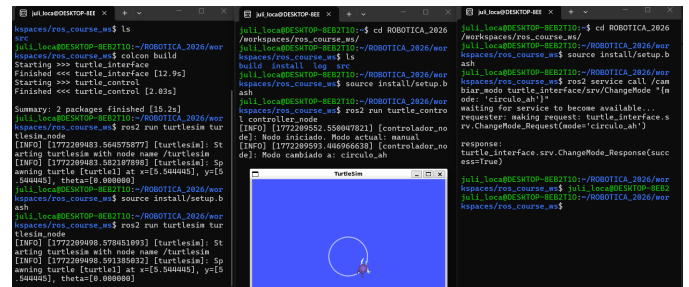


Fig. 6. Modo círculo antihorario

Horario

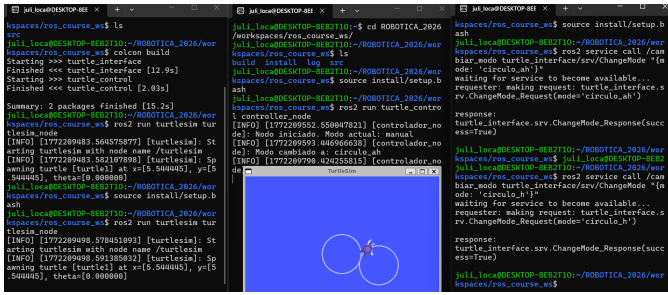


Fig. 7. Modo circulo horario

C. Modo Trayectoria

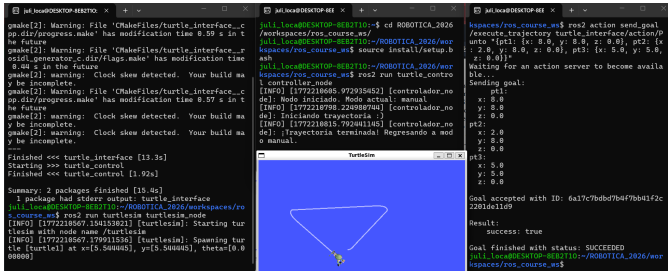


Fig. 8. Modo trayectoria

III. CONCLUSIÓN

REFERENCES